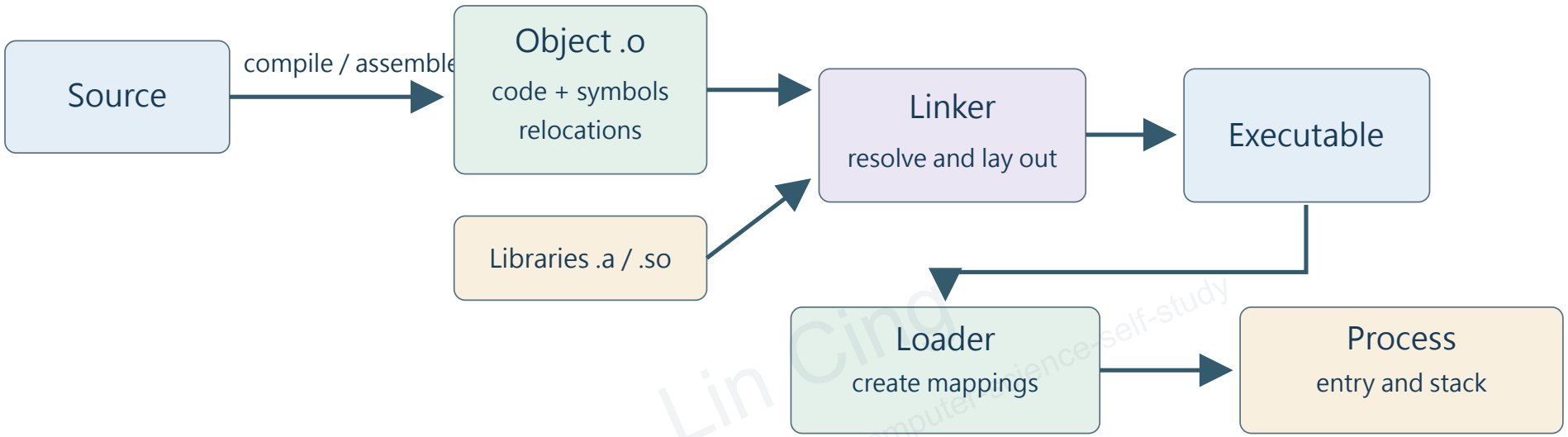
Five boundaries between source and machine state

Compilers produce assembly or objects. Assemblers encode instructions, directives, and labels into code, symbols, and relocations. Linkers bind cross-object names and assign addresses. Loaders establish process mappings. CPUs execute ISA operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Five boundaries between source and machine state (continued)



A parseable object need not execute correctly. Successful linking does not prove ABI compliance. Check legal fields, unique symbol bindings, correct relocations, segment permissions, and preserved caller/callee state at their respective boundaries.

The MIPS model

Use pre-Release-6 MIPS32, four-byte instructions, one branch/jump delay slot, and jal saving ra=PC+8. A simplified model without a delay slot may use PC+4 instead; the conventions cannot be mixed. Examples show nop slots and assume no load delay. Select matching SPIM delayed-branch settings; its services are not Linux's MIPS syscall ABI.

Addresses are 32-bit; integer results wrap modulo 2^32 when specified. Branches sign-extend imm 16 and multiply by four. J-targets concatenate PC+4's high nibble, the 26-bit field, and 00. This teaching object model states addends explicitly and is not a complete ELF parser.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.1 Instructions, directives, and macros

`addu $t0,$t1,$t2` is a machine instruction. `.text` selects a section, `.word` allocates data, and `.align n` aligns to 2^n bytes in this syntax. The CPU does not execute directives.

`move $s0,$s1` can expand to `addu $s0,$s1,$zero`; `la` may require two or more instructions depending on addressing. Macros expand syntax without runtime call frames. Binary size follows expanded instructions, data, and padding, not source lines times four. Assembly helps boot/exception/ABI boundaries; lower-level spelling alone does not prove speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.2 Two-pass assembler data

Pass one keeps a separate location counter per section and records each label's section/offset. Instructions, data, and alignment advance exact sizes. Pass two uses identical expansion and size rules. A known PC-relative reference within the same section can be encoded from its offset difference. A reference that still needs a final section base or external symbol produces a relocation. Different pseudo lengths between passes corrupt later labels; one expanded intermediate representation avoids guessing twice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A complete location-counter trace

All shown instructions are already four-byte machine instructions; initial text offset is zero.

```
.text
main: addiu $t0,$zero,0
loop: addiu $t0,$t0,1
      bne   $t0,$a0,loop
      nop
done: jr    $ra
      nop
.data
msg:  .asciiz "Hi"
.align 2
value: .word 0x1234abcd
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A complete location-counter trace (continued)

Event	Before	After
main instruction	text 0	text 4
loop instruction	text 4	text 8
bne, nop	text 8	text 16
done:jr, nop	text 16	text 24
msg plus NUL	data 0	data 3
align 2	data 3	data 4, one padding byte
value	data 4	data 8

Symbols are main=(text, 0), loop=(text, 4), done=(text, 16), msg=(data, 0), value=(data, 4). Linking later supplies section bases.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Branch encoding and relocation invariance

For text base 0x00400000, bne is 0x00400008 and loop 0x00400004:

$$imm = (target - (PC + 4))/4 = (0x00400004 - 0x0040000c)/4 = -2 = 0xfffe.$$

Opcode 5, rs 8, and rt 4 give

$$(5 \ll 26) \mid (8 \ll 21) \mid (4 \ll 16) \mid 0xfffe = 0x1504fffe.$$

Moving the whole section by delta adds the same delta to target and PC, so their difference is invariant. Layout changes or cross-section targets require recalculation, signed 16-bit range, and alignment checks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rewriting an out-of-range branch

In the delay-slot model, suppose `beq $s0,$s1,far` is at 0x00400000 with an original `nop` delay slot, and `far=0x00430000`. Its displacement is (0x00430000−0x00400004)/4=49151, beyond the signed 16-bit maximum 32767.

```
0x00400000  bne  $s0,$s1,skip
0x00400004  nop                # branch delay slot
0x00400008  j    far
0x0040000c  nop                # jump delay slot
0x00400010  skip:  ...         # original fall-through code
```

When equal, `bne` falls through and `j` reaches far; when unequal, the branch reaches skip after its slot. The `bne` displacement is (0x10−0x04)/4=3. This `j` requires far and its PC+4 to share the same 256 MiB region; otherwise use a reachable veneer or an indirect jump. The longer sequence changes layout, so later addresses and references must be recomputed.

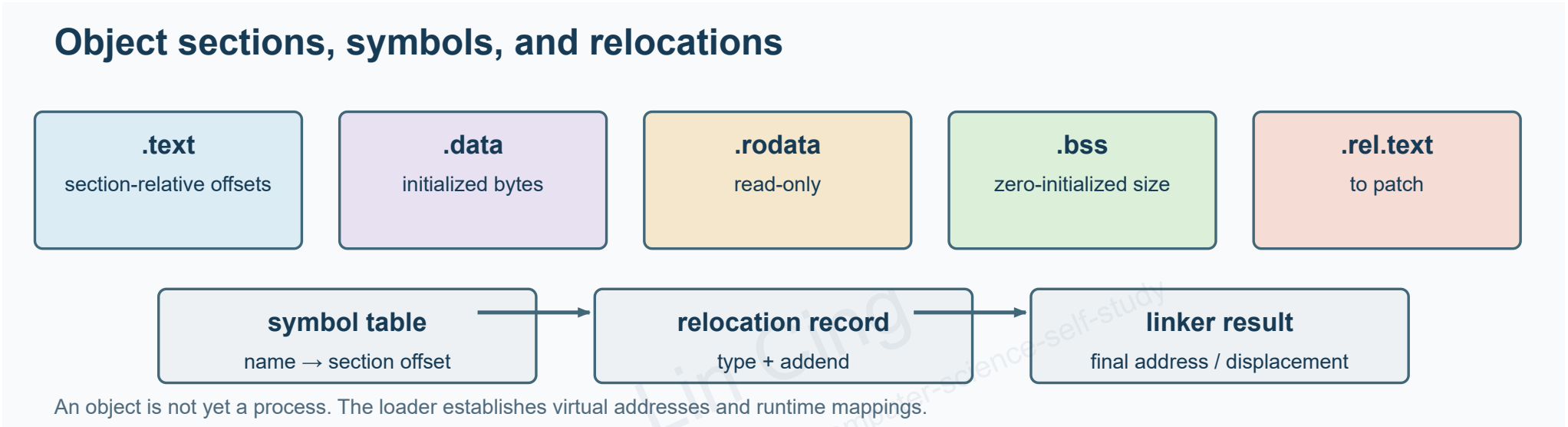
Macro expansion can clobber its input

An unsafe multiply-by-three macro expands to `addiu $t0,$zero,3; mult $arg,$t0; mflo $v0`. If arg is t0 with original 7, the first instruction overwrites it and produces 9 rather than 21. Specify clobbers and preserve inputs before overwrite, or select nonconflicting temporaries.

Internal labels also need unique expansion names. Conditional assembly selects emitted code at assembly time; it is not a runtime branch. Omitted code has no execution cost, but conditions change layout and symbols.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An object is not a memory snapshot



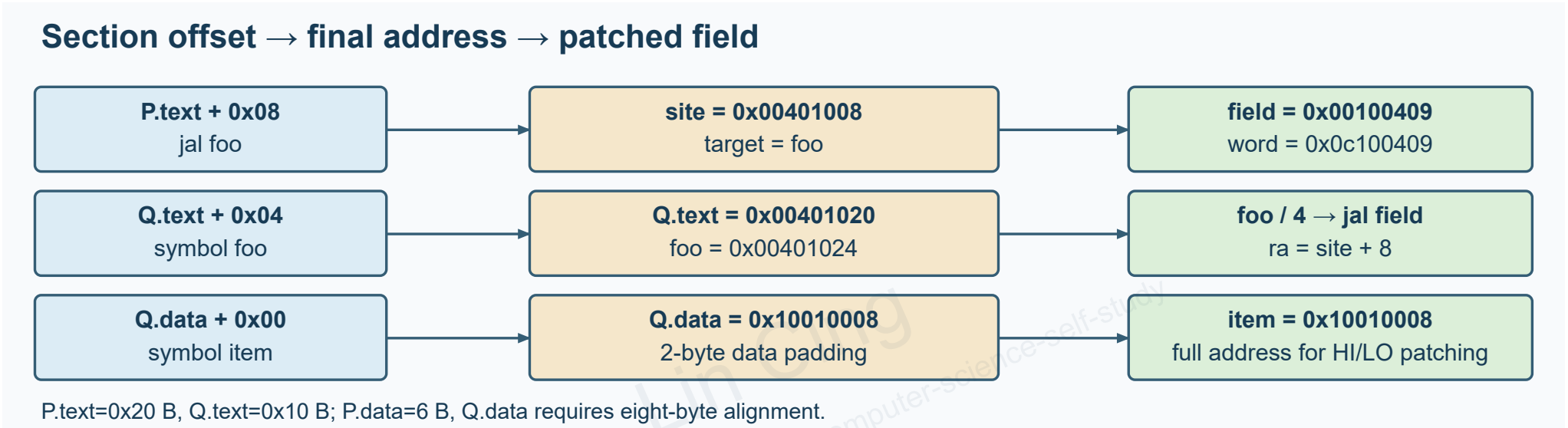
Text holds code, data initialized writable values, rodata constants, and bss zero-filled allocation size, usually without equal zero bytes in the file. Symbols also carry section, value, binding, and type. Relocations identify site, symbol, type, and how to obtain the addend, whether record-stored or encoded in place. A zero placeholder alone does not mean no repair is required. Debug information is not ordinary runtime load/store data.

Symbol-resolution scope

Local loop labels in different objects can coexist. Duplicate strong globals normally cause an error; undefined references need a legal definition, library, or runtime binding. Weak/common visibility follows format/toolchain rules rather than arbitrary selection. These examples use unique strong definitions. Name ambiguity must be resolved before relocation can compute a target.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.3 Merging two objects



P.text size 0x20, Q.text 0x10, P.data 6 bytes, Q.data 4 bytes aligned to eight. Text base 0x00401000, data 0x10010000, input order P then Q.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

B.3 Merging two objects (continued)

Input	Final base	Half-open range
P.text	0x00401000	[0x00401000, 0x00401020)
Q.text	0x00401020	[0x00401020, 0x00401030)
P.data	0x10010000	[0x10010000, 0x10010006)
padding	0x10010006	two bytes
Q.data	0x10010008	[0x10010008, 0x1001000c)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every field of a jal relocation

Q's foo=(text, 4) becomes 0x00401024; item=(data, 0) becomes 0x10010008. Add a symbol offset to its own input section's final base, not one common executable base.

P.text offset 8 is site 0x00401008. Target foo 0x00401024 is aligned and shares PC+4's high nibble.

$$field = (0x00401024 \gg 2)\ \&\ 0x03ffffff = 0x00100409, \quad word = (3 \ll 26) \mid field = 0x0c100409.$$

CPU concatenation reconstructs 0x00401024. jal writes ra 0x00401010 and executes site+4's nop slot first. A target in another 256 MiB region requires a legal veneer/indirect sequence or a range error; truncating 26 bits cannot fix it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

High/low relocation and sign extension

For V=0x1234abcd, lui/ori uses high 0x1234 and zero-extended low 0xabcd. For lui/addiu, that low is −21555, so

$$H = ((V + 0x8000) \gg 16)\ \&\ 0xffff = 0x1235, \qquad (0x1235 \ll 16) - 21555 = 0x1234abcd.$$

Using 0x1234 with addiu yields 0x1233abcd, off by 0x10000. Real HI16/LO16 relocations also obey ABI addend/pair rules; this final-value arithmetic does not implement every relocation format.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A left-to-right archive scan

Define a linker that always includes objects, then extracts needed archive members repeatedly to a fixed point within that archive, but never automatically revisits earlier archives. main.o references f; F.o defines f and references g; G.o defines g.

`main.o libF.a libG.a` updates unresolved sets {f}→{g}→{} and succeeds. Reversing libraries sees only f while at libG, extracts nothing, and later libF leaves g undefined. Actual linkers can provide group/rescan options; this is the declared model rather than a C-language rule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

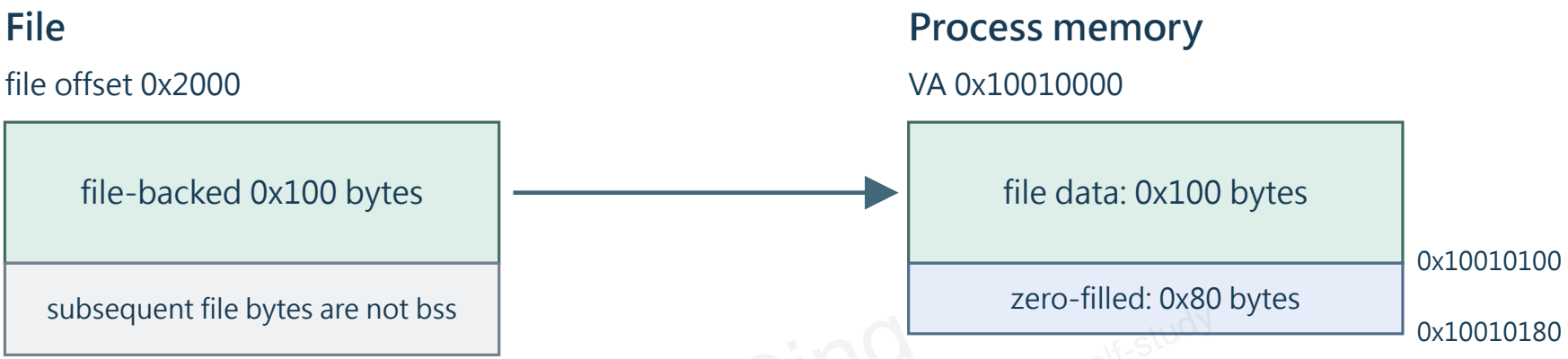
Dynamic linking and a GOT pointer

Static linking includes selected library code; dynamic linking binds shared-object references at startup or first use. Processes can map one readonly backing page at different VAs without sharing writable globals.

An abstract GOT slot starts unresolved. The loader finds foo 0x70001234 and installs that pointer; later indirect calls read it. Lazy binding can first invoke a resolver then update the slot. Actual stubs/registers follow their ABI. ASLR changes bases; PIC uses relative references/tables to reduce text patching. Matching names alone do not guarantee version/ABI compatibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.4 Loader file size and memory size



memory size = 0x180; mappings need not copy each byte eagerly.

A writable segment has file offset 0x2000, VA 0x10010000, file size 0x100, memory size 0x180, page size 0x1000. Map the first 0x100 bytes and zero the next 0x80; do not take unrelated following file bytes as bss.

B.4 Loader file size and memory size (continued)

Segments can combine sections with common permissions. Loaders primarily follow executable load metadata rather than section names. Text may be R-X, data RW-, and guard pages unmapped. Runtime setup precedes main; ELF/PIE/interpreter details depend on the OS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.5 Byte order and alignment

At 0x100, the value 0x12345678 has big-endian bytes 12, 34, 56, 78 or little-endian 78, 56, 34, 12. Byte placement changes, not the register value or bit positions within an instruction field.

Strict MIPS lw requires low address bits 00; 0x102 is unaligned. Packet formats need explicit widths, byte order, and padding; native struct dumps are not portable protocols. Typical stack/heap growth sketches are ABI/OS choices rather than universal layouts.

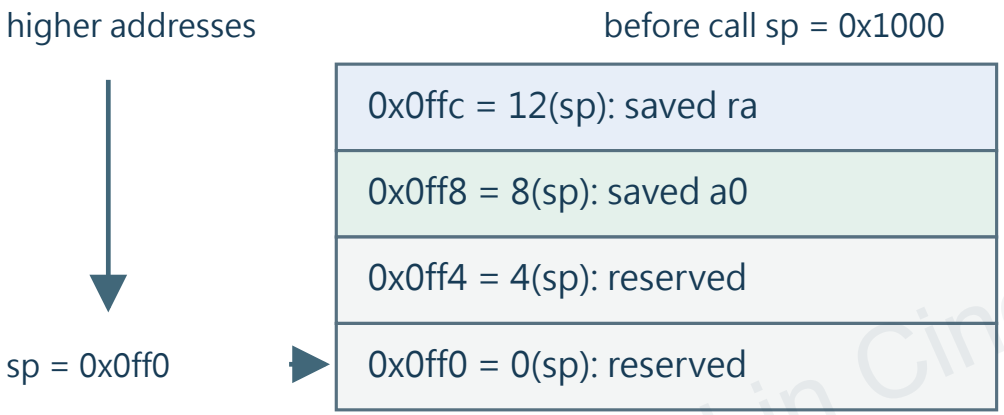
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.6 Caller and callee state

Arguments often use a0..a3, returns v0/v1, t-registers caller-saved, and s-registers callee-saved. A callee modifying s0 must restore its incoming value. A caller needing old a0 after a call must preserve it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16-byte frame: addresses and saved slots



Example: load first, then release 16 bytes

After changing sp, recalculate offsets

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

This recursion example uses a teaching frame, not a complete o32 nonleaf layout. Full ABIs can require outgoing argument space, alignment, and other saves. ISA-correct code with the wrong calling convention can still corrupt its caller.

The complete sum_to procedure

Require n≥0, a representable signed 32-bit sum, aligned entry sp, and enough stack. addu wraps modulo 2^32; the range precondition makes that result equal to the mathematical sum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The complete sum_to procedure (continued)

```
sum_to:
  addiu $sp,$sp,-16
  sw    $ra,12($sp)
  sw    $a0,8($sp)
  beq   $a0,$zero,base
  nop
  addiu $a0,$a0,-1
  jal   sum_to
  nop
  lw    $t0,8($sp)
  addu  $v0,$v0,$t0
  j     done
  nop
base:
  addu  $v0,$zero,$zero
done:
  lw    $ra,12($sp)
  addiu $sp,$sp,16
  jr    $ra
  nop
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The complete sum_to procedure (continued)

Each transfer has its slot. Reload this activation's n from its frame after recursion; neither a0 nor a pre-call t0 is guaranteed to survive.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Actual recursive frames and results

Start at sp 0x1000 and call sum_to(3); the base case also allocates.

n	Current sp	Saved n address	Saved ra address	Return v0
3	0x0ff0	0x0ff8	0x0ffc	6
2	0x0fe0	0x0fe8	0x0fec	3
1	0x0fd0	0x0fd8	0x0fdc	1
0	0x0fc0	0x0fc8	0x0fcc	0

Unwinding computes 0, 0+1=1, 1+2=3, 3+3=6 and restores sp 0x1000. Four simultaneous frames use 64 bytes. Restore with the current base before releasing it. Old storage remains addressable with recalculated offsets or an old-base register, but blindly retaining offsets after moving sp reads the wrong addresses.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inductive recursion correctness

The contract returns v0=n(n+1)/2 and restores sp/callee-saved state. At n=0, the base returns zero and restores ra/sp. For positive n, save this activation's n/ra in its nonoverlapping frame. By induction, the child returns (n−1) n/2 and restores the parent's sp. Reload n, add it, and obtain n(n+1)/2. The epilogue restores ra/sp. Decreasing nonnegative n terminates.

Stack capacity and representability are premises. Negative inputs, unbounded recursion, and stack overflow lie outside the contract. Running n=3 alone does not prove it generally.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.7 Exceptions are not ordinary ABI calls

Synchronous faults arise from instructions; interrupts arrive externally. A handler can interrupt any point with caller-saved values live, so saving only callee-saved registers is insufficient. Hardware provides recovery PC/status; software saves what it clobbers, identifies cause, handles/acknowledges sources, and returns according to recovery policy. Uncleared device state can immediately retrigger. Nested handlers need separate contexts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EPC with and without a branch delay slot

A recoverable non-slot fault has BD=0 and EPC at the faulting instruction. Repairing a page then retrying normally returns there. A completed syscall can instead advance under a stated policy.

If a branch is 0x00400020 and its slot load 0x00400024 faults, BD=1 and EPC 0x00400020 point to the branch. EPC+4 alone loses its decision; preserve/reconstruct branch and slot semantics or correctly reexecute them. Precise state preserves older effects and prevents forbidden faulting/younger effects. Interrupt priority does not reorder synchronous faults past older instructions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.8 Polling an MMIO receiver

Define a teaching device with control 0xffff0000, data 0xffff0004, ready bit 0. Reading data consumes a byte and clears ready. MMIO is uncached with the required platform ordering; these addresses are not a universal SPIM guarantee.

```
lui   $t0,0xffff
poll:
lw    $t1,0($t0)
andi  $t1,$t1,1
beq   $t1,$zero,poll
nop
lw    $v0,4($t0)
andi  $v0,$v0,0xff
```

Control returns 0, 0, 1. There are three control reads and one data read. Moving the data read into the branch slot executes it even when not ready, changing device state. Keep the nop.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.9 SPIM service semantics

SPIM print-int uses v0=1 and a0=integer; print-string uses v0=4 and a0=NUL-terminated address; exit uses v0=10. Service semantics come from the simulator table rather than instruction encoding. A service may clobber/result registers as specified. Real OS ABI numbers and argument rules differ.

```
addiu $v0,$zero,1
addiu $a0,$zero,42
syscall
addiu $v0,$zero,10
syscall
```

Neither syscall occupies a delay slot here. The simulator can implement the service directly rather than execute an identical real-OS handler. Observing output 42 checks this program's declared service behavior; it does not establish that a physical MIPS system uses the same exception vector or kernel ABI.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.10 Instruction-semantic checks

Instructions	Operation and boundary
add/addu	same low 32-bit sum; add traps on signed overflow, addu does not
addiu	sign-extended immediate, no signed-overflow trap
andi/ori	zero-extended immediate
slt/sltu	signed/unsigned comparison can differ on identical bits
lb/lbu	sign/zero-extend an eight-bit value
lw/sw	base+signext immediate, aligned words in this model
beq/bne	compare GPRs, target=PC+4+(signext imm<<2), execute slot
j/jal	PC+4 high nibble; jal also writes ra=PC+8
jr	register target with a slot

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.10 Instruction-semantic checks (continued)

0xffffffff means −1 under slt and 4294967295 under sltu. Signedness follows the operation/data contract, not a tag carried by bits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.11 Trace an incorrect address to its boundary

Wrong-function faults suggest symbol resolution/link maps. A constant off by 0x10000 suggests signed-low/high-pair arithmetic. Wrong post-call registers suggest ABI/slot state. Reversed bytes suggest serialization/endianness.

Assembler listings expose source/offset/word; object dumps expose symbols/relocations; link maps expose final placement; execution traces expose PC/register changes. Check the relevant field instead of assuming a compiler defect.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.12 Model boundaries for the exercises

Recompute layouts, negative low halves, archive dependencies, and recovery behavior under explicit versions. Do not memorize one hex output. Full ISAs also include FP, coprocessor, and system operations. A reference entry must be read for semantics, exceptions, alignment, and delay rules as well as opcode.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Revised Fourth Edition, Appendix B. B.2–B.9, printed B-10–B-44, PDF 815–849; link figure B.3.1 at B-19, PDF 824; instruction reference B.10 at B-45–B-80, PDF 850–885. These object/numeric/recursion examples use their stated teaching models; full platform interoperability requires the actual ABI.

References and attribution

David A. Patterson and John L. Hennessy, Computer Organization and Design, Revised Fourth Edition; Lin Cing, handwritten computer organization notes; computer organization and structure (CSO) course slides.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study